

编译原理

8. Basic Blocks and Traces

rainoftime.github.io

浙江大学

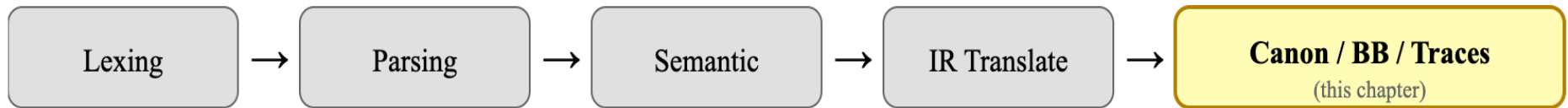
计算机科学与技术学院

课程内容

1. Introduction
2. Lexical Analysis
3. Parsing
4. Abstract Syntax
5. Semantic Analysis
6. Activation Record
7. Translating into Intermediate Code
- 8. Basic Blocks and Traces**
9. Instruction Selection
10. Liveness Analysis
11. Register Allocation
13. Garbage Collection
14. Object-oriented Languages
18. Loop Optimizations

Where Are We?

- The compiler pipeline so far:



- **Input:** IR Trees (with ESEQ, nested CALL, two-way CJUMP)
- **Output:** A clean, linear list of statements ready for instruction selection
- Three phases:

Linearize → Basic Blocks → Trace Schedule

Outline



Canonical Form



Step I: Canonical Trees



Step II & III: Taming Conditional Branches

1. Canonical Form

Motivation

- The operators of the *Tree* language are chosen carefully to match the capabilities of most machines
- However, the IR tree still has several *mismatches* with real machine code:
 1. CJUMP has two target labels → real machines *fall through* on false
 2. ESEQ nodes inside expressions → side effects make evaluation order matter
 3. Function CALLs
 - CALL nodes inside expressions → same problem (they write to RV register)
 - Nested CALL in arguments of another CALL → clobbers parameter registers

Mismatches: Trees vs. Machine Code

1. **CJUMP** can jump to two labels

- Real machines' conditional jump instructions **fall through** to the next instruction if the condition is false (e.g., JZ, JNZ)

Tree IR:

```
CJUMP(>, i, 10, trueLabel, falseLabel)
```

Assembly:

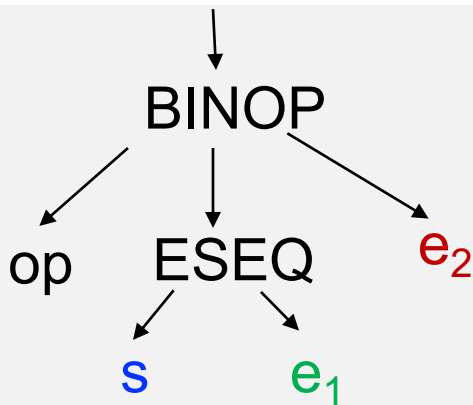
```
CMP i, 10           ; Compare i with 10
JG trueLabel        ; Jump if greater
(falseLabel...)     ; Must be immediately after
```

The Tree IR's two-way jump must be converted to a specific sequence that follows hardware constraints.

Mismatches: Trees vs. Machine Code

2. ESEQ nodes inside expressions

- Different orders of evaluating the subtrees yield different results
- But it is useful to be able to evaluate the subexpressions of an expression in any order



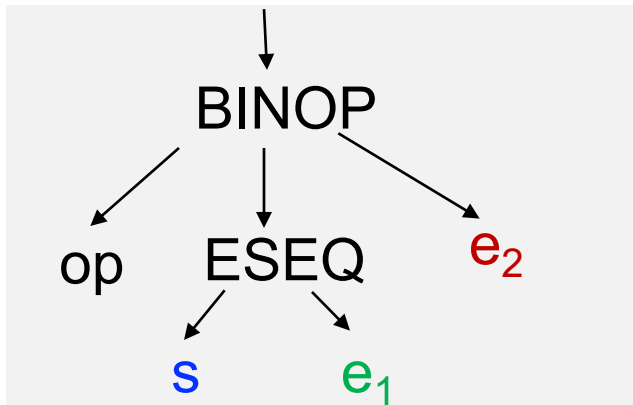
ESEQ(s, e): The statement **s** is evaluated for side effects, then **e** is evaluated for a result.

如果计算**s**的时候有side-effect，那就会导致谁先做谁后做结果不一样(为什么)

Mismatches: Trees vs. Machine Code

2. ESEQ nodes inside expressions

- Different orders of evaluating the subtrees yield different results
- But it is useful to be able to evaluate the subexpressions of an expression in any order



BINOP(PLUS,
TEMP a,
ESEQ(MOVE(TEMP a, 5), v))

- MOVE有副作用: 修改了临时变量a的值 !
- 也就是说, ESEQ(s, e1)可能改变了e2!

Evaluate e2 first or not?

Mismatches: Trees vs. Machine Code (Cont.)

3. CALL nodes

– E.g., when trying to put arguments into a fixed set of formal-parameter registers

– e.g., $\text{CALL}(f, [e1, \text{CALL}(g, [e2, \dots])])$

e1: 传给f的首个参数
e2: 传给g的首个参数

$\text{CALL}(g, [e2, \dots])$ 可能会“影响” $e1$ ，体现在不同层面：

语句副作用问题：

- $\text{CALL}(g, [e2, \dots])$ 除了返回值外，可能还会修改全局变量、改变堆内存, etc.
- 如果 $e1$ 依赖于这些可能被 g 修改的状态，那么 $e1$ 的值在 g 调用前后可能不同

Mismatches: Trees vs. Machine Code (Cont.)

3. CALL nodes

- E.g., when trying to put arguments into a fixed set of formal-parameter registers
- e.g., $\text{CALL}(f, [e1, \text{CALL}(g, [e2, \dots])])$

$\text{CALL}(g, [e2, \dots])$ 可能会“影响” $e1$ ，体现在不同层面：

寄存器使用冲突：

- 在固定参数寄存器的调用约定中， $e1$ 的值应放在第一个参数寄存器中
- 但 g 函数的返回值（作为 f 的第二个参数）通常会放在返回值寄存器中
- 在许多调用约定中，第一个参数寄存器和返回值寄存器可能是同一个！

例：ARM64（AArch64）架构调用约定：

- 使用寄存器X0-X7传递整数和指针参数，第一个参数在X0。
- 返回值也是通过X0寄存器返回

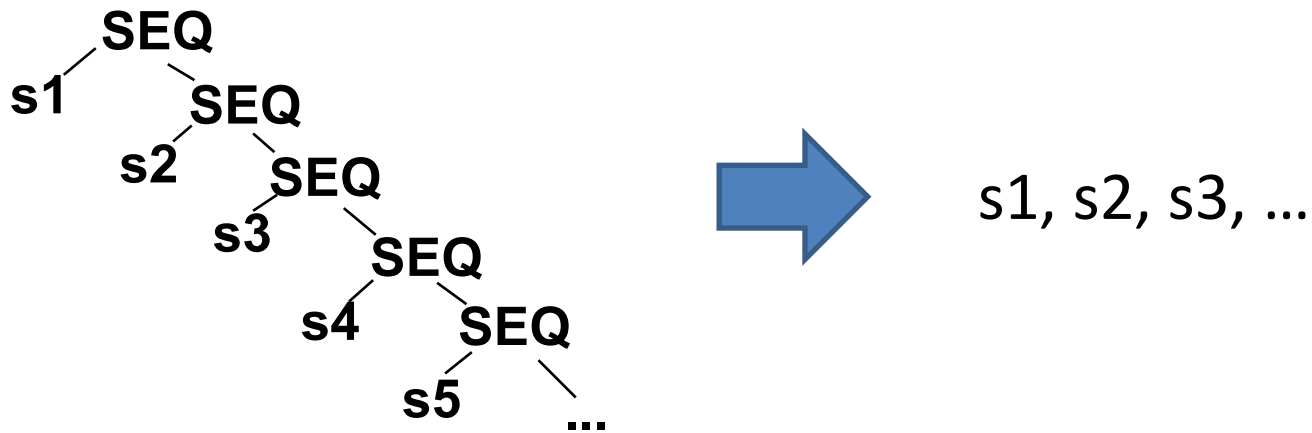
Mismatches: Trees vs. Machine Code

- **Problems**
 1. CJUMP can jump to two labels
 2. ESEQ nodes within expressions are inconvenient
 3. CALL
- Intermediate code has general tree form
 - Easy to generate from AST,
 - But hard to translate directly to assembly
- Assembly code is **a sequence of statements!**

Idea: Transform the IR to a canonical form

Canonical Form/Trees?

- Some characteristics of **canonical form**
 - All statements brought up to top level of tree



- A function is just a sequence of all statements:
s1,s2,s3,s4,s5,...
- Can translate to assembly more directly!

把statements全部移上去，最后就会和汇编比较接近(语句序列!)

Three-Stage Transformation

- 1. Linearize:** Transform trees into a list of *canonical trees* without SEQ or ESEQ nodes
- 2. Basic blocks:** Group statements into sequences with no internal jumps/labels
- 3. Trace schedule:** Order basic blocks so every CJUMP is followed by its false label

1. Linearize

Remove ESEQ, lift CALL
→ flat stm list

2. Basic Blocks

Group into blocks:
LABEL ... JUMP/CJUMP

3. Trace Schedule

Order blocks so CJUMP
falls through to false label

2. To Canonical Trees (Linearization)

- **eliminate ESEQs**
 - Basic rules
 - Commutativity
- move CALLs to top level
- eliminate SEQs

Stage I: To *Canonical Trees*

- A tree is **canonical** if it satisfies:
 1. **No SEQ or ESEQ**
 2. **The parent of each CALL is either**
 - **EXP(...)** : for calls used for their side effects
 - **MOVE(TEMP t, ...)**: for calls that produce values
- To perform the transformation
 1. **Eliminate ESEQ**
 2. **Move CALLs to top level**
 3. **Eliminate SEQs (turn into linear lists)**

Eliminate ESEQ: Linearization Rules

1. Identify patterns where ESEQ appears in expressions
2. Apply **rewrite rules** to push ESEQs upward
3. Eventually, convert ESEQs to SEQs when they reach statement position

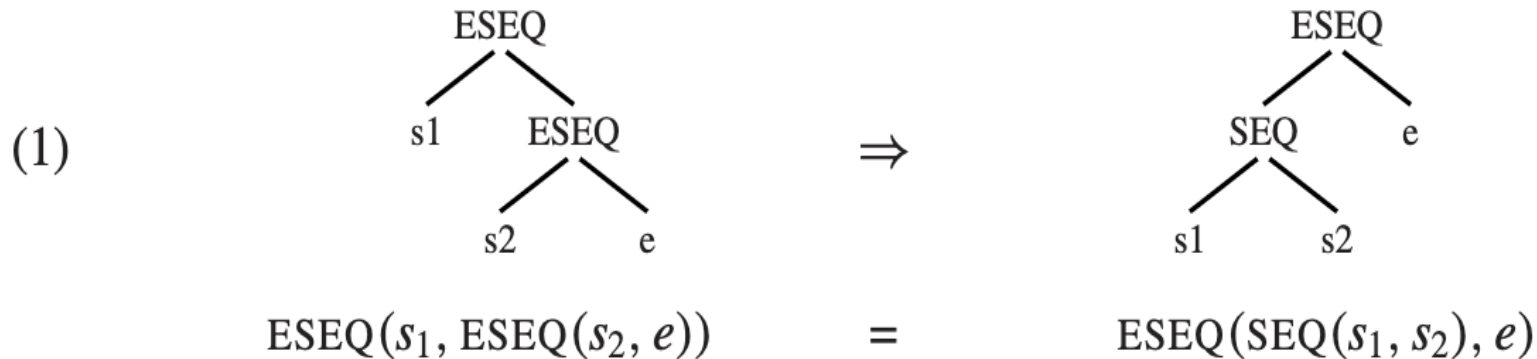


Lift ESEQ nodes higher and higher in the tree,
until they can become SEQ nodes.

Linearization Rules for ESEQ

- A simple ESEQ transformation rule

$$\text{ESEQ}(s1, \text{ESEQ}(s2, e)) \Rightarrow \text{ESEQ}(\text{SEQ}(s1, s2), e)$$

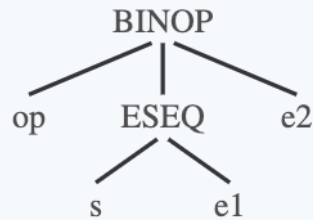


This combines the statement parts $s1$ and $s2$ into a single SEQ node.

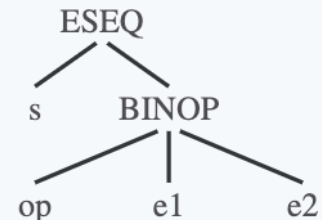
More Linearization Rules for ESEQ

$ESEQ(s1, ESEQ(s2, e))$	$\Rightarrow$	$ESEQ(SEQ(s1,s2), e)$
$BINOP(op, ESEQ(s, e1), e2)$	$\Rightarrow$	$ESEQ(s, BINOP(op, e1, e2))$
$MEM(ESEQ(s,e1))$	$\Rightarrow$	$ESEQ(s, MEM(e1))$
$JUMP(ESEQ(s, e1))$	$\Rightarrow$	$SEQ(s, JUMP(e1))$
$CJUMP(op, ESEQ(s, e1), e2, l1,l2)$	$\Rightarrow$	$SEQ(s, CJUMP(op, e1, e2, l1,l2))$

(2)



$\Rightarrow$



$BINOP(op, ESEQ(s, e1), e2) = ESEQ(s, BINOP(op, e1, e2))$

$MEM(ESEQ(s, e1)) = ESEQ(s, MEM(e1))$

$JUMP(ESEQ(s, e1)) = SEQ(s, JUMP(e1))$

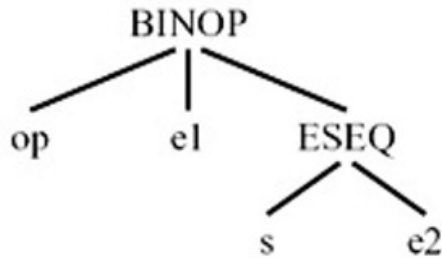
$CJUMP(op, ESEQ(s, e1), e2, l1, l2) = SEQ(s, CJUMP(op, e1, e2, l1, l2))$

2. To Canonical Trees (Linearization)

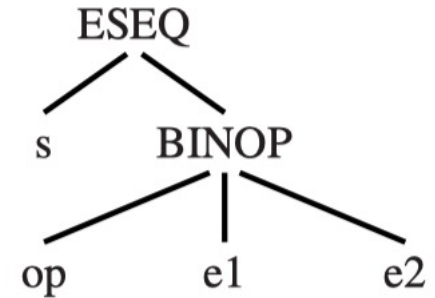
- **eliminate ESEQs**
 - Basic rules
 - **Commutativity (重点)**
- move CALLs to top level
- eliminate SEQs

ESEQ in Second Operand (Complex Case)

BINOP(op, e1, ESEQ(s, e2))



Can we ?



$\text{BINOP}(op, e1, \text{ESEQ}(s, e2))$

$\text{ESEQ}(s, \text{BINOP}(op, e1, e2))$

We **cannot** simply pull **s** above **e1** —
s might have side effects that change **e1**'s value!

- **The problem:** **s** may have side effects that affect **e1**!

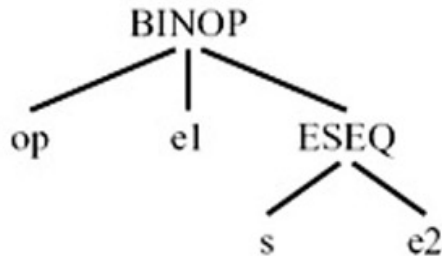
Example:

s = MOVE(MEM(x), y)

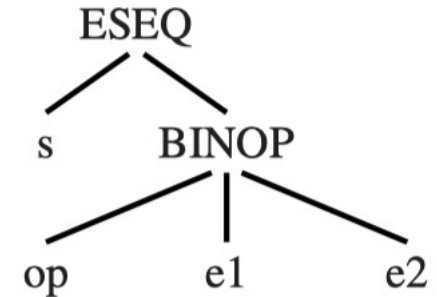
e1 = MEM(x)

ESEQ in Second Operand (Complex Case)

BINOP(op, e1, ESEQ(s, e2))



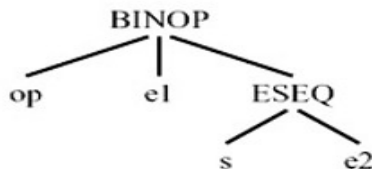
Can we ?



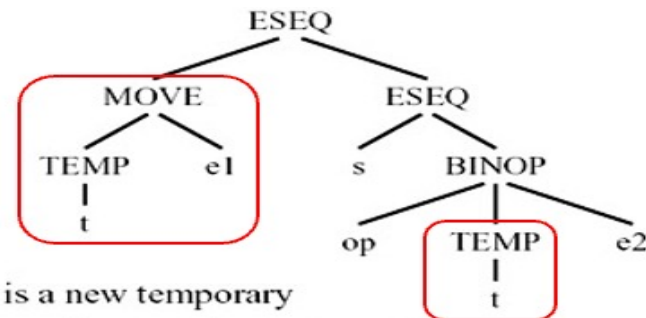
$\text{BINOP}(op, e_1, \text{ESEQ}(s, e_2))$

$\text{ESEQ}(s, \text{BINOP}(op, e_1, e_2))$

- **The solution:** use a temporary to store value of **e1**



$\text{BINOP}(op, e_1, \text{ESEQ}(s, e_2))$



t is a new temporary
 $\text{ESEQ}(\text{MOVE}(\text{TEMP } t, e_1),$
 $\text{ESEQ}(s, \text{BINOP}(op, \text{TEMP } t, e_2)))$

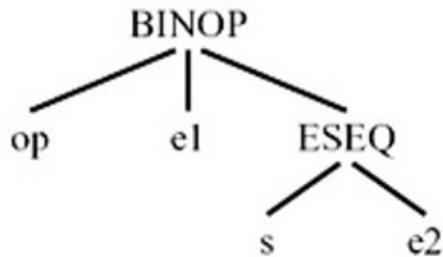
Commutativity and its Effects on Linearizations

- **Commutativity**: statement **s** and expression **e** can commute if **s** does not affect the value of **e**
 - E.g., consider MOVE (MEM(t1), 3) and MEM(t2)
 - If $t1 == t2$, then MOVE (MEM(t1), 3) and MEM(t2) do not commute
- What if statement **s** and expression **e** do not commute?
 - We may need **new temporary locations to store intermediate results** to get canonical trees

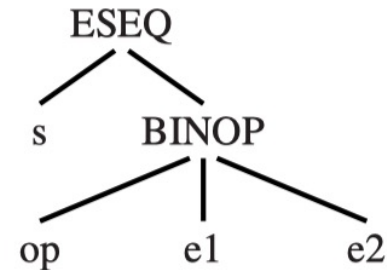
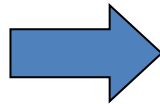
Recap: Effect of Commutativity on BINOP

BINOP(op, e1, ESEQ(s, e2))

If statement **s** and expression **e** commute (无影响)

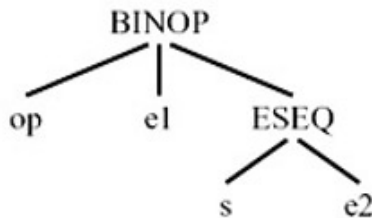


$\text{BINOP}(op, e_1, \text{ESEQ}(s, e_2))$

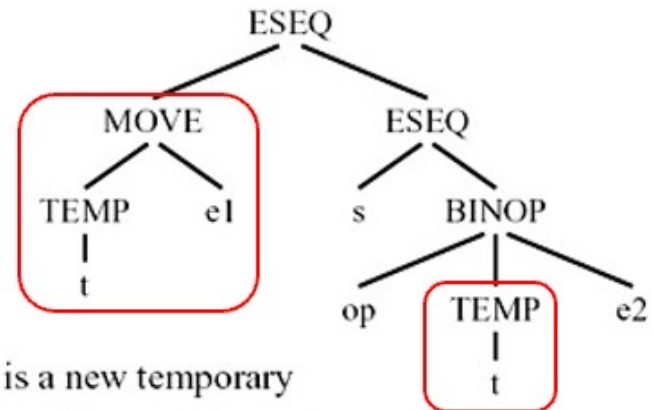
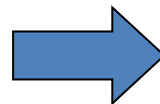


$\text{ESEQ}(s, \text{BINOP}(op, e_1, e_2))$

If statement **s** and expression **e** DO NOT commute (有影响)



$\text{BINOP}(op, e_1, \text{ESEQ}(s, e_2))$



t is a new temporary
 $\text{ESEQ}(\text{MOVE}(\text{TEMP } t, e_1),$
 $\text{ESEQ}(s, \text{BINOP}(op, \text{TEMP } t, e_2)))$

More Example: Effect of Commutativity

If **s**, **e1** commute, we have (more concise) rules

- **BINOP(op, e1, ESEQ(s, e2)) $\Rightarrow$ ESEQ(s, BINOP(op, e1, e2))**
- **CJUMP(op, e1, ESEQ(s, e2), I1,I2) $\Rightarrow$
SEQ(s, CJUMP(op, e1, e2, I1,I2))**

Else, use the following rules (that contain new temporaries)

- **BINOP(op, e1, ESEQ(s, e2))**
 $\Rightarrow$ ESEQ(MOVE(TEMP t, e1),
ESEQ(s, BINOP(op, TEMP t, e2)))
- **CJUMP(op, e1, ESEQ(s, e2), I1, I2)**
 $\Rightarrow$ SEQ(MOVE(TEMP t, e1),
SEQ(s, CJUMP(op, TEMP t, e2, I1,I2)))

Deciding Commutativity

Whether a statement **s** commutes with an expression **e**?

- **Problem:** commutativity is hard to know statically
- Make a **conservative/safe approximation**
 - $\text{commute}(s, e) = \text{True}$ if **s** and **e** definitely do commute
 - $\text{commute}(s, e) = \text{False}$ otherwise

Using commutation analysis allows us to generate simpler, more efficient code when safe (e.g., the previous page).

Deciding Commutativity

Whether a statement **s** commutes with an expression **e**?

- **Problem:** commutativity is hard to know statically
- Make a **conservative/safe approximation**
 - $\text{commute}(\text{s}, \text{e}) = \text{True}$ if they definitely do commute
 - $\text{commute}(\text{s}, \text{e}) = \text{False}$ otherwise
- **A naïve strategy** to estimate whether a statement commutes with an expression:
 1. A **constant** commutes with any statement
 2. An **empty statement** commutes with any expression.
 3. Anything else is assumed not to commute

Deciding Commutativity (不要掌握)

- Rules for BINOP and MOVE rely on the interchanging the order of a lowered statement **s** and an expression **e**
 - This can be done safely when the statement cannot alter the value of the expression
- **Two conditions that **s** and **e** cannot be interchanged**
 1. The statement could **change** the value of **a temporary variable used** by the expression
 2. The statement could **change** the value of **a memory location used by** the expression.

Deciding Commutativity (不要求掌握)

How to check the above two conditions?

- **Temporaries**: It is easy to determine whether the statement updates a temporary used by the expression, because temporaries have unique names.
- **Memory**: It is much harder because two memory locations can be **aliases**!
 - The statement **s** uses a memory location $MME(e1)$ as a destination
 - The expression reads from the memory location $MEM(e2)$ $e1$ might have the same value as $e2!!$

More precise (still conservative) approximation: use **alias analyses**!

Summary of ESEQ Rewriting Identities

#	Pattern	Rewrites To	Condition
1	$\text{ESEQ}(s_1, \text{ESEQ}(s_2, e))$	$\text{ESEQ}(\text{SEQ}(s_1, s_2), e)$	Always
2	$\text{BINOP}(\text{op}, \text{ESEQ}(s, e_1), e_2)$	$\text{ESEQ}(s, \text{BINOP}(\text{op}, e_1, e_2))$	Always
3	$\text{BINOP}(\text{op}, e_1, \text{ESEQ}(s, e_2))$	$\text{ESEQ}(\text{MOVE}(t, e_1), \text{ESEQ}(s, \text{BINOP}(\text{op}, t, e_2)))$	s, e_1 don't commute
4	$\text{BINOP}(\text{op}, e_1, \text{ESEQ}(s, e_2))$	$\text{ESEQ}(s, \text{BINOP}(\text{op}, e_1, e_2))$	s, e_1 commute

Analogous rules apply to MEM, JUMP, CJUMP, and MOVE.

2. To Canonical Trees (Linearization)

- eliminate ESEQs
- **move CALLs to top level**
- eliminate SEQs

Move CALLS to Top Level

- **Problem:** All functions return results in the *same* register RV.
- Example: BINOP(PLUS, CALL(f, []), CALL(g, []))
If function calls f() and g() both return values in register RV:
 1. CALL(f, []) executes, result in RV
 2. CALL(g, []) executes, overwrites RV
 3. PLUS can't access f()'s result anymore!

Move CALLS to Top Level

- **The Solution: Function Result Preservation**
- **Idea:** assign each return value immediately into a fresh temporary register

```
CALL(fun, args) =>  
    ESEQ(MOVE(TEMP t, CALL(fun, args)), TEMP t)
```

最终目标: The parent of each CALL is either EXP(...) or MOVE(TEMP t, ...)

```
BINOP(PLUS,  
      CALL(f, []),  
      CALL(g, []))
```



```
SEQ(MOVE(TEMP t1, CALL(f, [])),  
    SEQ(MOVE(TEMP t2, CALL(g, [])),  
        BINOP(PLUS, TEMP t1, TEMP t2)))
```

Example: CALL Rewriting — Step by Step

- Original

```
BINOP(PLUS, CALL(f,[]), CALL(g,[]))
```

- After CALL rewriting

```
BINOP(PLUS,  
      ESEQ(MOVE(TEMP t1, CALL(f,[])), TEMP t1),  
      ESEQ(MOVE(TEMP t2, CALL(g,[])), TEMP t2))
```

- After ESEQ elimination

```
SEQ(MOVE(TEMP t1, CALL(f,[])),  
    SEQ(MOVE(TEMP t2, CALL(g,[])),  
        BINOP(PLUS, TEMP t1, TEMP t2)))
```

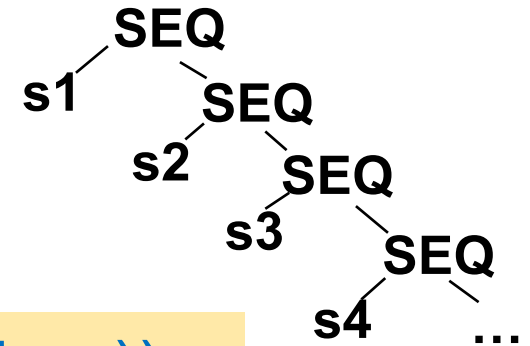
2. To Canonical Trees (Linerization)

- eliminate ESEQs
- move CALLs to top level
- **eliminate SEQs**

A Linear List of Statements

- After applying the above rules, we obtain

$\text{SEQ}(\text{SEQ}(\text{SEQ}(\dots, s_x), s_y), s_z)$



- Next, we repeatedly apply the rule:

$\text{SEQ}(\text{SEQ}(a, b), c) = \text{SEQ}(a, \text{SEQ}(b, c))$

- And obtain a statement of the form $\text{SEQ}(s_1, \text{SEQ}(s_2, \dots, \text{SEQ}(s_{n-1}, s_n) \dots))$, which can be just considered as a list of statements: $s_1, s_2, \dots, s_{n-1}, s_n$
(None of the s_i contain SEQ/ESEQ nodes)

Example: Fully Linearized Code

```
if x > 0 then
    (print("Positive"); x := x + 1)
else
    (print("Non-positive"); x := 0)
```



// List of statements with no SEQ or ESEQ nodes

CJUMP(GT, TEMP x, CONST 0, L1, L2)

LABEL L1

MOVE(TEMP t1, CALL(NAME print, [CONST "Positive"]))

MOVE(TEMP x, BINOP(PLUS, TEMP x, CONST 1))

JUMP(NAME L3)

LABEL L2

MOVE(TEMP t2, CALL(NAME print, [CONST "Non-positive"]))

MOVE(TEMP x, CONST 0)

LABEL L3

Summary: Building Canonical Trees

Step	What It Does	Result
do_exp / do_stm	Pull ESEQs out	All SEQs at top
CALL rewriting	Wrap each CALL in ESEQ(MOVE(TEMP t,...), TEMP t)	CALLs under MOVE/EXP only
linear	Flatten nested SEQs into a list	Flat stm list, no SEQ/ESEQ

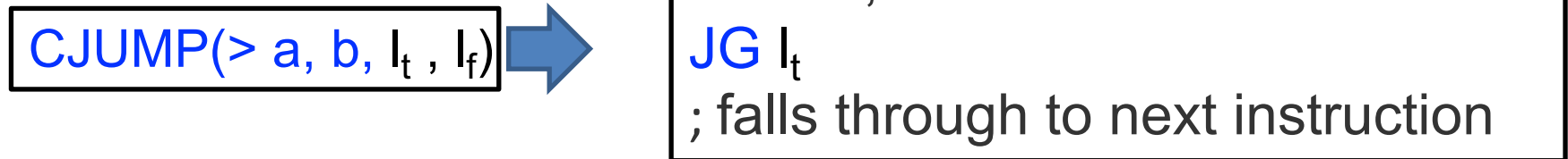
Output: a linear list of *canonical* statements. Ready for basic-block formation.

3. Taming Conditional Branches

- **Basic Blocks**
- **Traces**

Taming Conditional Branches

- **Problem of CJUMP:** **NO** counterpart for **two-way branch** on most machines
- **Goal:** rearrange the trees so that **CJUMP(cond, l_t , l_f)** is immediately followed by **LABEL(l_f)**



- **Solution:** two-stages approach
 1. Form a list of canonical trees into **basic blocks**
 2. Order the basic blocks into **traces**

Transforming to Canonical Form

- 1. Linearize:** Transform trees into a list of *canonical trees* without SEQ or ESEQ nodes
- 2. Basic blocks:** Group statements into sequences with no internal jumps/labels
- 3. Trace schedule:** Order basic blocks so every CJUMP is followed by its false label



Control Flow

- **Control flow** = the sequencing of instructions, ignoring data values.
 - We don't know data → can't predict conditional branches → assume either way.
 - Non-branch instructions have "boring" control flow (just fall through).
 - We can **group** sequences of non-branch instructions together.
- This grouping gives us **basic blocks**

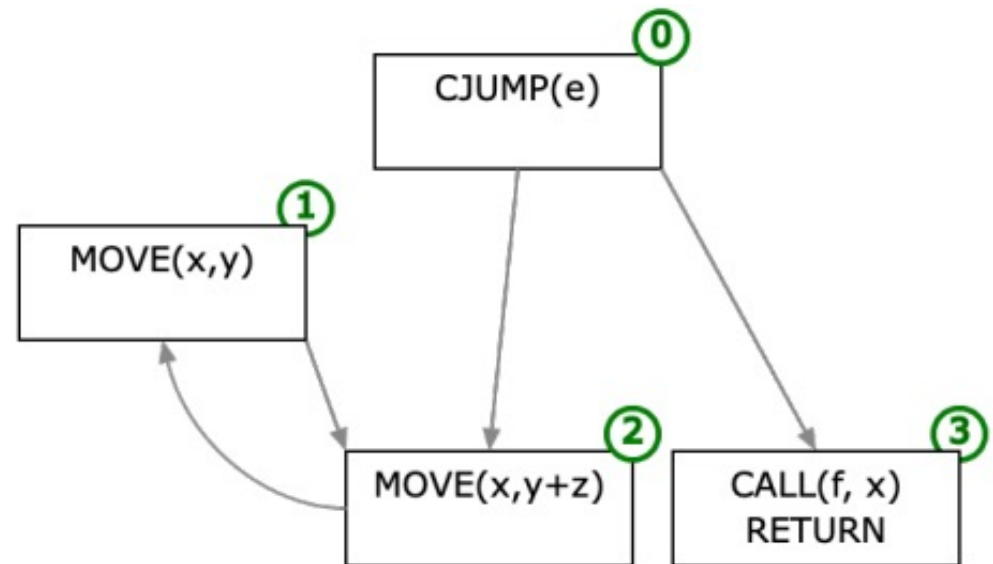
Basic Blocks

- A **basic block** is a sequence of statements that
 1. Always **entered at the beginning** (first statement is a LABEL).
 2. Always **exited at the end** (last statement is a JUMP or CJUMP).
 3. Contains **no other** LABEL, JUMP, or CJUMP inside.

Example: Basic Blocks

- **Control flow graph** (CFG): nodes are basic blocks and edges are jumps between them
 - In some contexts, the node of a CFG is a single statement (to be discussed in register allocation)

L0:	CJUMP(e, L2, L3)
L1:	MOVE(x, y)
L2:	MOVE(x, y + z) JUMP(L1)
L3:	CALL(f, x) RETURN



Basic Block Construction

Scan the linearized statement list from beginning to end:

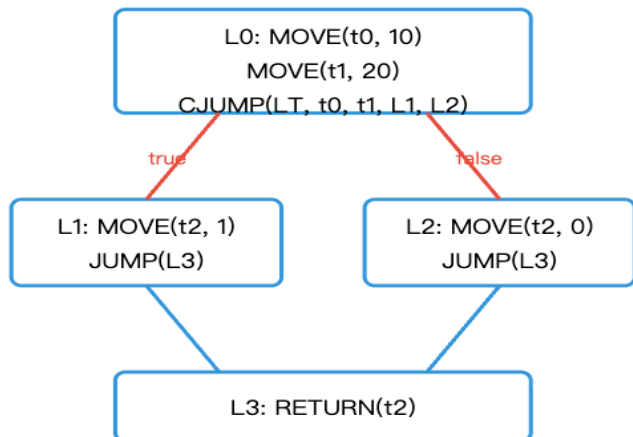
1. When a **LABEL** is found, **start a new block**
2. When a **JUMP** or **CJUMP** is found, **end the current block**

Fixups:

1. Block doesn't start with a **LABEL** → **add one**
2. Block doesn't end with **JUMP** or **CJUMP** → **add a JUMP to the next block**

Example: Forming Basic Blocks

L0: 语句序列
 MOVE(t0, 10)
 MOVE(t1, 20)
 CJUMP(LT, t0, t1, L1, L2)
L1:
 MOVE(t2, 1)
 JUMP(L3)
L2:
 MOVE(t2, 0)
L3:
 RETURN(t2)



// 基本块 B0

L0:
 MOVE(t0, 10)
 MOVE(t1, 20)
 CJUMP(LT, t0, t1, L1, L2)

// 基本块 B1

L1:
 MOVE(t2, 1)
 JUMP(L3)

// 基本块 B2

L2:
 MOVE(t2, 0)
 JUMP(L3) // 隐式添加的跳转

// 基本块 B3

L3:
 RETURN(t2)

3. Taming Conditional Branches

- **Basic Blocks**
- **Traces**

Transforming to Canonical Form

- 1. Linearize:** Transform trees into a list of *canonical trees* without SEQ or ESEQ nodes
- 2. Basic blocks:** Group statements into sequences with no internal jumps/labels
- 3. Trace schedule:** Order basic blocks so every CJUMP is followed by its false label



Basic Block Reordering

- Basic blocks can be **reordered arbitrarily** without changing program semantics.
- Every block ends with an explicit jump to its successor — the physical ordering doesn't matter for correctness.
- But the ordering *does* matter for:
 - **Performance:** Unnecessary jumps waste cycles.
 - **CJUMP fall-through:** Machine conditional jumps fall through on false — we want CJUMP's false label to be the next block.

→ This motivates **trace scheduling**.

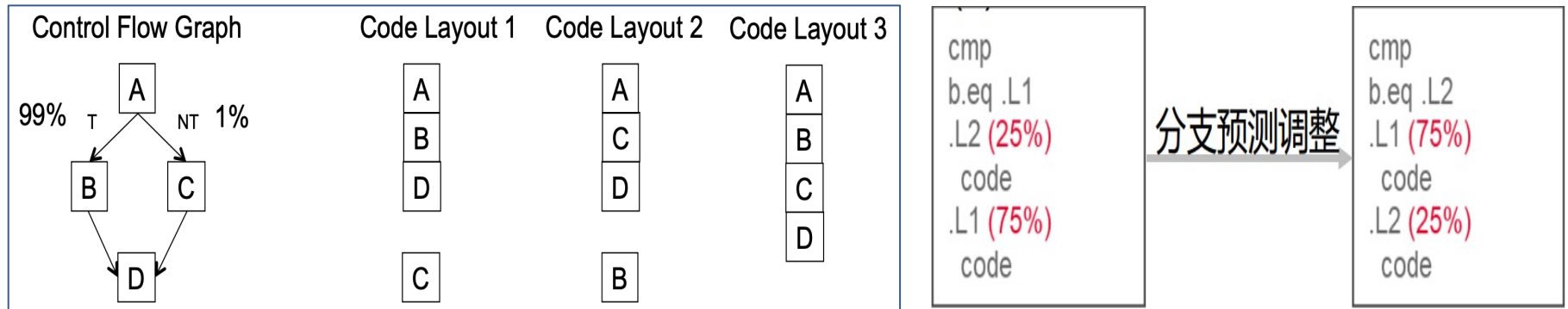
Basic Block Reordering

- We can optimize the *nature* and *number* of jumps (and other aspects):

Benefit	Explanation	Impact
CJUMP Structure	Arrange CJUMPs to be immediately followed by their false labels	Direct mapping to hardware conditional jumps
Reduced Jumps	Place the target block right after unconditional jumps	Fewer executed jumps, better performance
Instruction Cache Locality	Place frequently executed code sequences together	Higher instruction cache hit rate
Branch Prediction	Create more predictable branch patterns	Fewer pipeline stalls, faster execution
...	...	...

Example: Basic Block Reordering (供了解)

- We can optimize basic block reordering for better instruction caching!



- 90%, 10%: the execution frequency (from dynamic profiling)
- Code Layout 1**: reduces fetch breaks, increases I-cache hit rate,...

Example: Basic Block Reordering (供了解)

- 现代 CPU 流水线深，**分支预测失败**代价高（ $\approx 10-20$ cycles）
- 编译器默认无法知道运行时哪条分支更「热」
- 注解让编译器把**热路径**布局为 fall-through，**冷路径**跳走

GCC / CLANG 底层

```
// 告知编译器 cond 几乎为真
if (__builtin_expect(cond, 1)) {
    hot_path();
} else {
    cold_path();
}
```

宏封装 (LINUX KERNEL 风格)

```
#define likely(x) \
    __builtin_expect(!(x), 1)
#define unlikely(x) \
    __builtin_expect(!(x), 0)

if (unlikely(err)) {
    handle_error();
}
```

C++20 属性

```
if (ok) [[likely]] {
    fast_path();
} else [[unlikely]] {
    slow_path();
}

// 也可用于 switch
switch (tag) {
    case 0: [[likely]]
        return handle0();
}
```

What is a Trace?

- A **trace** is a sequence of basic blocks that *could* be executed consecutively during a single run of the program.
 - A trace can include conditional branches (we just pick one direction).
 - A program has many overlapping traces.
 - We want a set of traces that **exactly covers** the program: each block in exactly one trace.
 - Fewer traces $\rightarrow$ fewer inter-trace jumps $\rightarrow$ less overhead.

Trace Scheduling

- **Trace scheduling**: generating a **covering set of traces**
 1. Each trace is loop free
 2. Each block must be in exactly one trace
- By ordering blocks into traces, we aim for:

Primary goal:

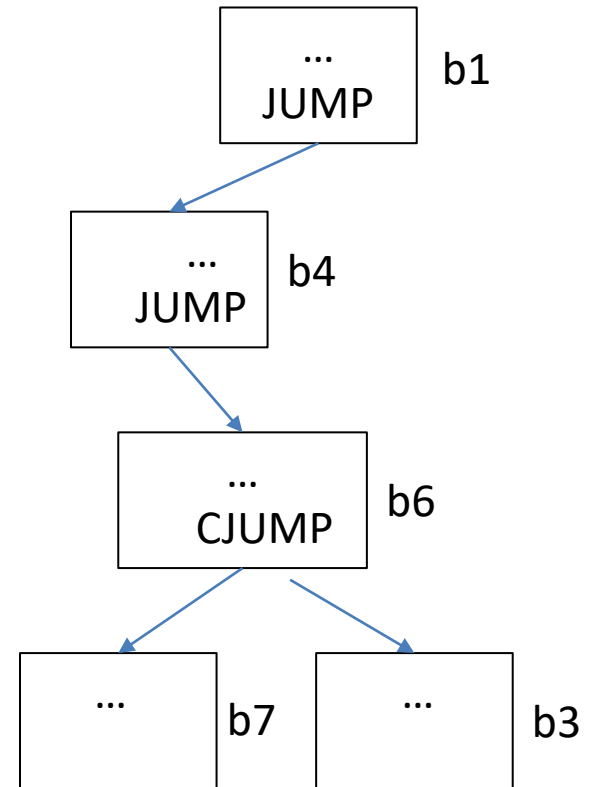
Every CJUMP(cond, lt, lf) is immediately followed by LABEL(lf).
→ Maps directly to a machine conditional branch.

Secondary goal:

Many JUMP(NAME L) are immediately followed by LABEL(L).
→ These jumps can be **deleted** (fall through).

Basic Rule for Generating ONE Trace

- Suppose block $b1$ ends with a **JUMP** to $b4$, and $b4$ has a JUMP to $b6$. Then, we can make the trace **$b1, b4, b6$** .
- Suppose $b6$ ends with a conditional jump **CJUMP** (**cond**, **$b7, b3$**). We append **$b3$** to our trace and continue with the rest of the trace after $b3$.
- The basic block $b7$ will be in some other trace.



Make sure that CJUMP(cond, l_t , l_f) is immediately followed by LABEL(l_f).

(Greedy) Trace Generation

```
Put all the blocks of the program into a list Q
While Q is not empty
    start a new (empty) trace, call it T
    remove the head element b from Q
    while b is not marked
        mark b; append b to the end of the current trace T;
        examine the successors of b
        if there is any unmarked successor c
            b ← c
    end the current trace T
```

Algorithm 8.3 or Tiger Book

- Start with some block and follow a chain of jumps, marking each block and appending it to the **current trace**
- When coming to a block whose successors are all marked, the generation of **one trace is finished** → Pick an unmarked block to start the **next trace**

(Greedy) Trace Generation

- **如何计算1个trace**

- 从某个basic block开始，往后继节点遍历，标记每个被访问的basic block并将其附加到当前trace中
- 当到达某basic block，其后继节点均已标记, 这个trace就算完了

- **如何计算新的trace**

- 选择未标记的basic block作为下一个trace的起点

- **全局终止条件**

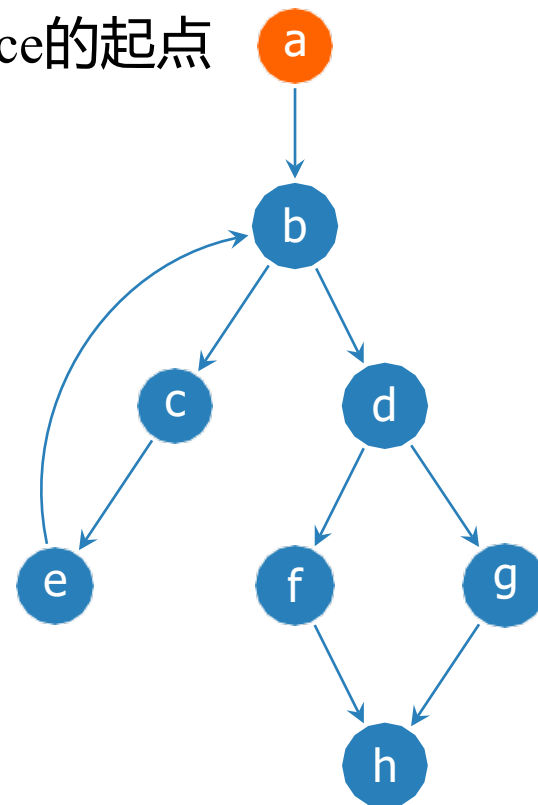
- 不断迭代、直到所有的basic blocks都被标记了

Example: Generating a Covering Set of Traces

- **Basic algorithm:** depth-first traversal of the CFG
 - 从某个basic block开始，往后继节点遍历，标记每个被访问的basic block并将其附加到当前trace中
 - 当到达某个basic block, 其后继节点均已标记, 这个trace就算完了
 - 选择一个未标记的basic block作为下一个trace的起点

Covering set of traces

1. {a}

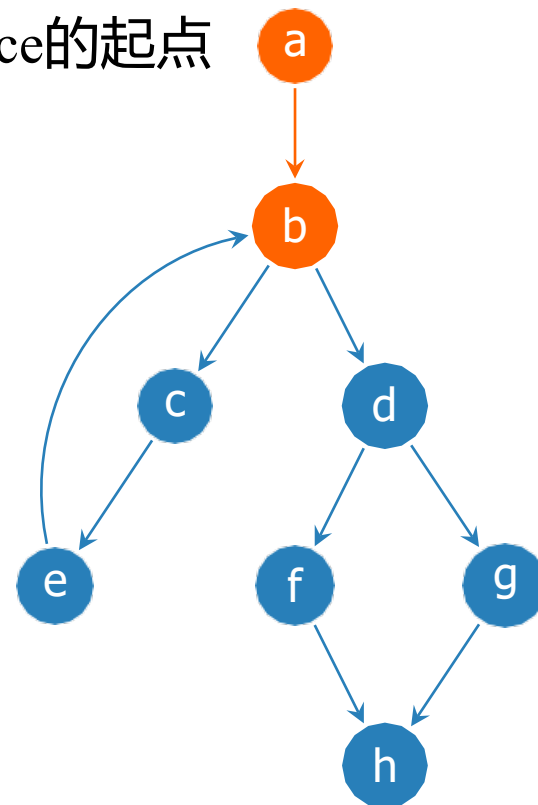


Example: Generating a Covering Set of Traces

- **Basic algorithm:** depth-first traversal of the CFG
 - 从某个basic block开始，往后继节点遍历，标记每个被访问的basic block并将其附加到当前trace中
 - 当到达某个basic block, 其后继节点均已标记, 这个trace就算完了
 - 选择一个未标记的basic block作为下一个trace的起点

Covering set of traces

1. $\{a, b\}$

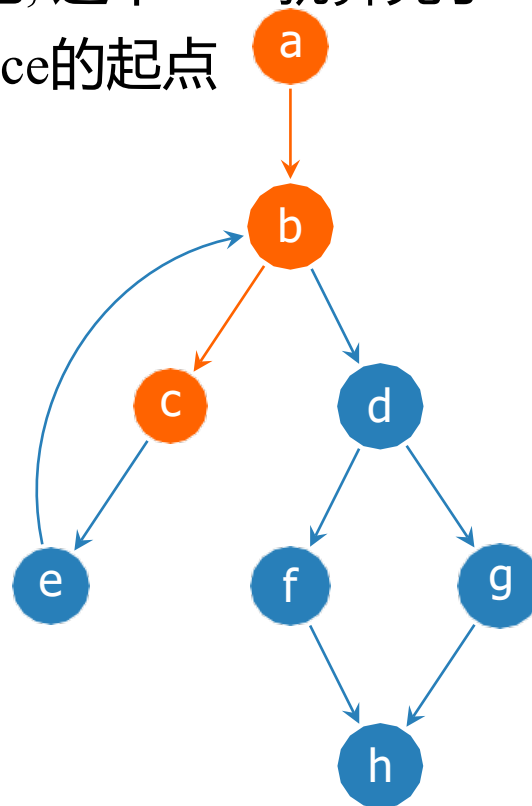


Example: Generating a Covering Set of Traces

- **Basic algorithm:** depth-first traversal of the CFG
 - 从某个basic block开始，往后继节点遍历，标记每个被访问的基本块并将其附加到当前trace中
 - 当到达某个basic block，其后继节点均已标记，这个trace就算完了
 - 选择一个未标记的基本块作为下一个trace的起点

Covering set of traces

1. {a, b, c}

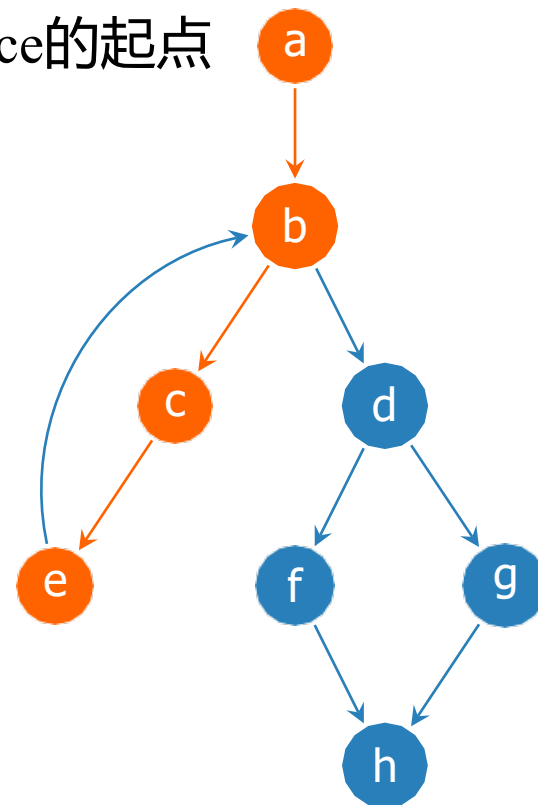


Example: Generating a Covering Set of Traces

- **Basic algorithm:** depth-first traversal of the CFG
 - 从某个basic block开始，往后继节点遍历，标记每个被访问的basic block并将其附加到当前trace中
 - 当到达某个basic block, 其后继节点均已标记, 这个trace就算完了
 - 选择一个未标记的basic block作为下一个trace的起点

Covering set of traces

1. {a, b, c, e}



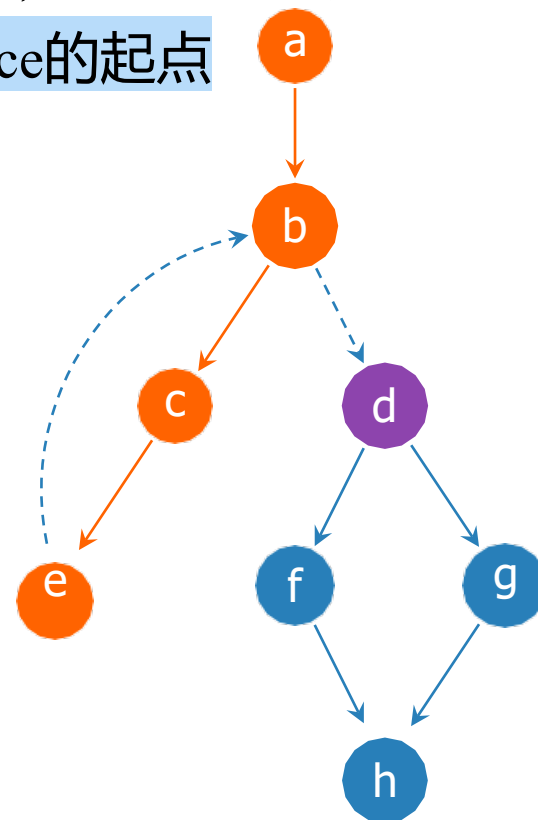
Example: Generating a Covering Set of Traces

- **Basic algorithm:** depth-first traversal of the CFG
 - 从某个basic block开始，往后继节点遍历，标记每个被访问的basic block并将其附加到当前trace中
 - 当到达某个basic block, 其后继节点均已标记, 这个trace就算完了
 - 选择一个未标记的基本块作为下一个trace的起点

Covering set of traces

1. {a, b, c, e}

2. {d}



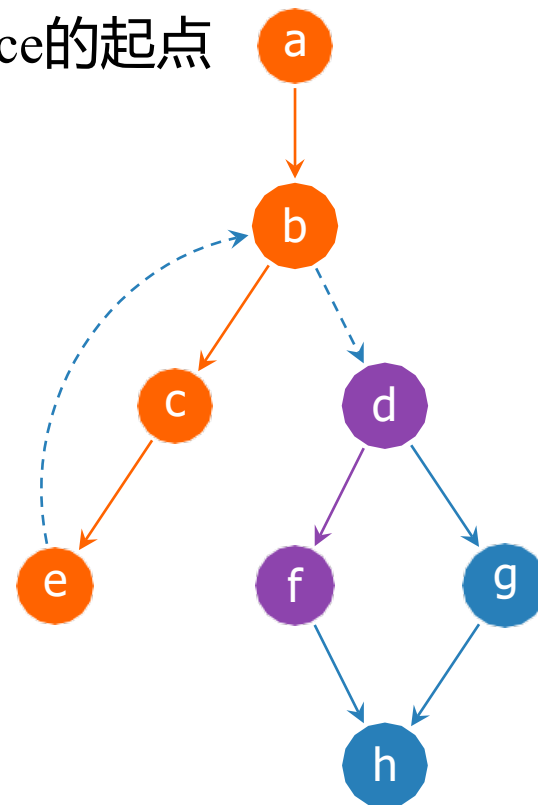
Example: Generating a Covering Set of Traces

- **Basic algorithm:** depth-first traversal of the CFG
 - 从某个basic block开始，往后继节点遍历，标记每个被访问的basic block并将其附加到当前trace中
 - 当到达某个basic block, 其后继节点均已标记, 这个trace就算完了
 - 选择一个未标记的basic block作为下一个trace的起点

Covering set of traces

1. {a, b, c, e}

2. {d, f}



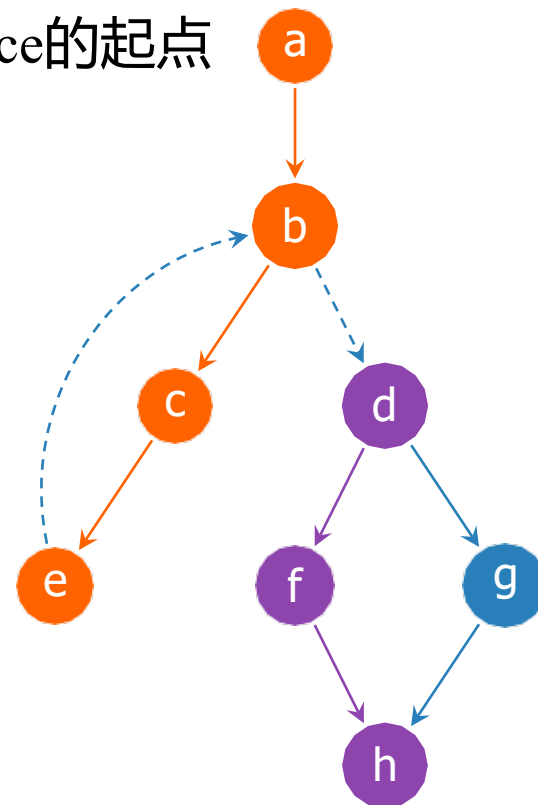
Example: Generating a Covering Set of Traces

- **Basic algorithm:** depth-first traversal of the CFG
 - 从某个basic block开始，往后继节点遍历，标记每个被访问的基本块并将其附加到当前trace中
 - 当到达某个basic block，其后继节点均已标记，这个trace就算完了
 - 选择一个未标记的基本块作为下一个trace的起点

Covering set of traces

1. {a, b, c, e}

2. {d, f, h}



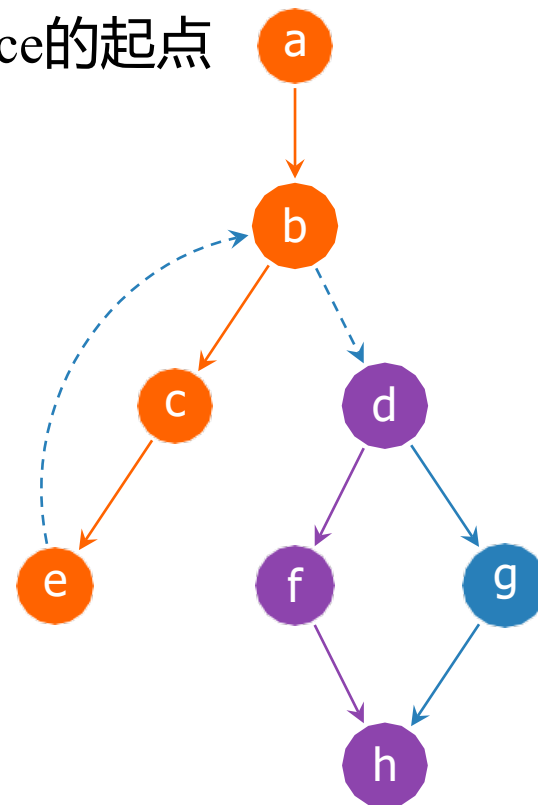
Example: Generating a Covering Set of Traces

- **Basic algorithm:** depth-first traversal of the CFG
 - 从某个basic block开始，往后继节点遍历，标记每个被访问的basic block并将其附加到当前trace中
 - 当到达某个basic block, 其后继节点均已标记, 这个trace就算完了
 - 选择一个未标记的basic block作为下一个trace的起点

Covering set of traces

1. {a, b, c, e}

2. {d, f, h}

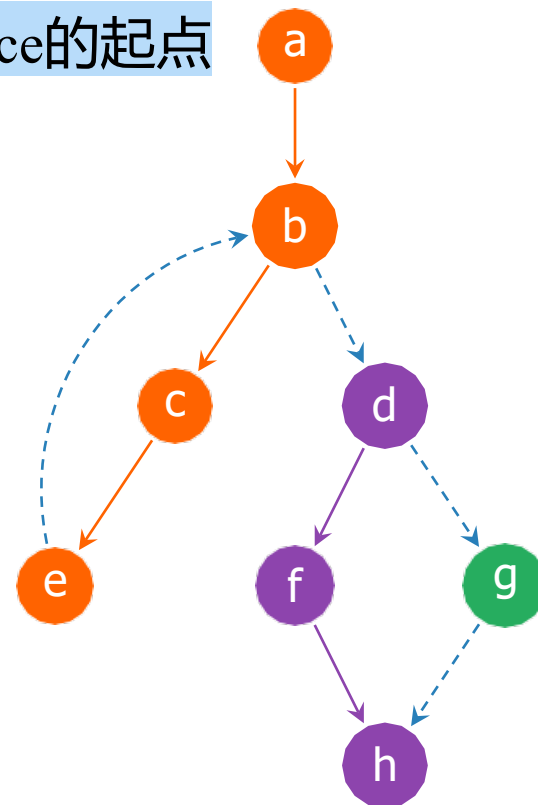


Example: Generating a Covering Set of Traces

- **Basic algorithm:** depth-first traversal of the CFG
 - 从某个basic block开始，往后继节点遍历，标记每个被访问的basic block并将其附加到当前trace中
 - 当到达某个basic block, 其后继节点均已标记, 这个trace就算完了
 - 选择一个未标记的basic block作为下一个trace的起点

Covering set of traces

1. {a, b, c, e}
2. {d, f, h}
3. {g}



Finishing Up (JUMP Consideration)

- After organizing code into traces, many (but not all) CJUMPs will be followed by their false labels.
- Need to handle the remaining cases to ensure every CJUMP “**fall through**”

Situation	Action
CJUMP followed by its false label	Leave alone ✓ (most common)
CJUMP followed by its true label	Swap true/false labels, negate condition
CJUMP followed by neither label	Invent new false label l'_f , insert: CJUMP(..., l_t , l'_f) LABEL(l'_f) JUMP(NAME l_f)

Finishing Up: Different CJUMP Cases

1. CJUMP immediately followed by its false label

- We let alone (there will be many of these).

```
CJUMP (>, x, y, Lt, Lf)  
LABEL Lf ...
```

→ Leave as is (ideal case)

Finishing Up: Different CJUMP Cases

1. **CJUMP immediately followed by its false label**

2. **CJUMP followed by its true label**

– Switch the true, false labels and negate the condition.

```
CJUMP (>, x, y, Lt, Lf)  
LABEL Lt ...
```

→ Negate condition and swap labels

```
CJUMP (<=, x, y, Lf, Lt)  
LABEL Lt ...
```

Finishing Up: Different CJUMP Cases

1. **CJUMP immediately followed by its false label**
2. **CJUMP followed by its true label**
3. **CJUMP(cond, a, b, lt, lf) followed by neither label**

```
CJUMP(>, x, y, Lt, Lf)  
...other code...
```

→ Add new label and jump

```
CJUMP(>, x, y, Lt, L'f)  
LABEL L'f  
JUMP(Lf)  
...other code...
```

Deleting Redundant JUMPs

- After trace ordering, many blocks look like:

... statements ...

JUMP(NAME L)

LABEL(L) ← the jump target is the very next instruction!

- These JUMPs are **redundant** — execution just falls through. **Delete them.**
- This is a simple peephole optimization that reduces code size and improves performance.

Discussion: Optimal Traces

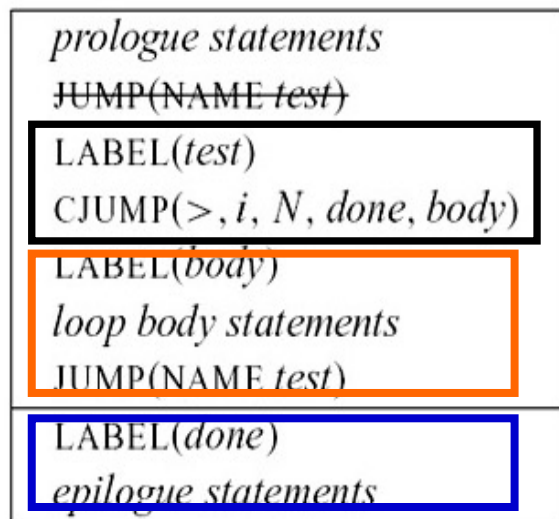
- The “Optimality” needs some criteria.
- Some guiding principles for trace selection
 1. **Hot Path Prioritization:** Place frequently executed blocks in the same trace
 2. **Locality Enhancement:** Keep related code together in memory
 3. **Jump Minimization:** Reduce the number of unconditional jumps in common execution paths
 4. ...

Example: Optimal Traces

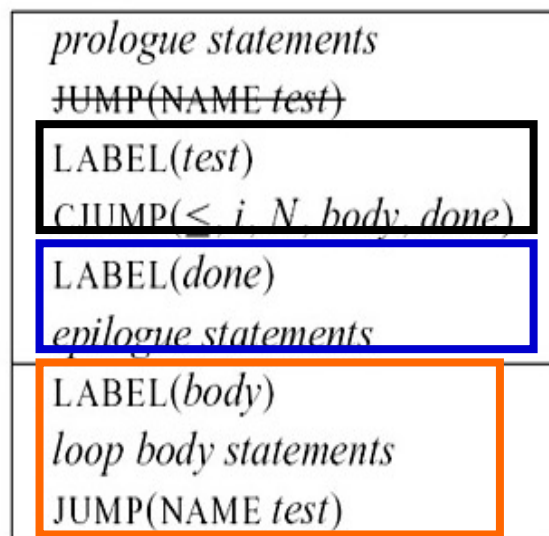
(a): While循环的每个迭代有一个CJUMP和一个JUMP

(b): 使用了不同traces, 但每个迭代仍有一个CJUMP和一个JUMP

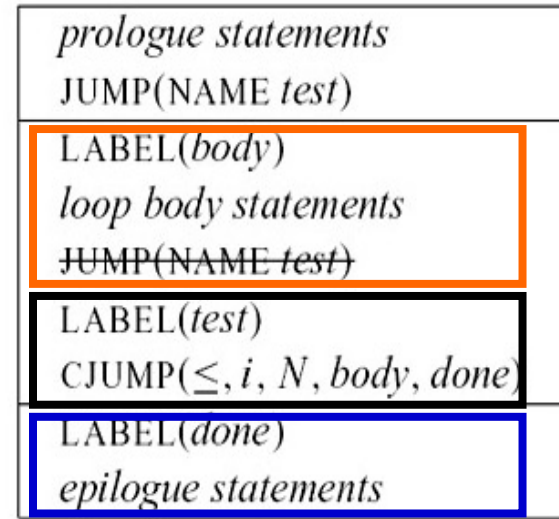
(c): 每个迭代都没有JUMP (虽然它在出口处增加了一次跳转) 🍌



(a)



(b)



(c)

Covering	Block Order	Jumps/Iteration
(a) Naive	prologue, test, body, done	1 CJUMP + 1 JUMP
(b) Swapped CJUMP	prologue, test, done, body	1 CJUMP + 1 JUMP
(c) Optimal	prologue, body, test, done	1 CJUMP only

Summary

- **Problem:** mismatches between tree code and machine instructions:
 - 1. CJUMP to two labels; machine conditionals fall through on false
 - 2. ESEQ and CALL order evaluation of subtrees matters (side-effects)
 - 3. CALL as argument to another CALL causes interference between register arguments
- **Approach:** three-stage transformation
 - To linear list of canonical trees without SEQ/ESEQ
 - To basic blocks with no internal jumps or labels
 - To traces with every CJUMP immediately followed by its false target

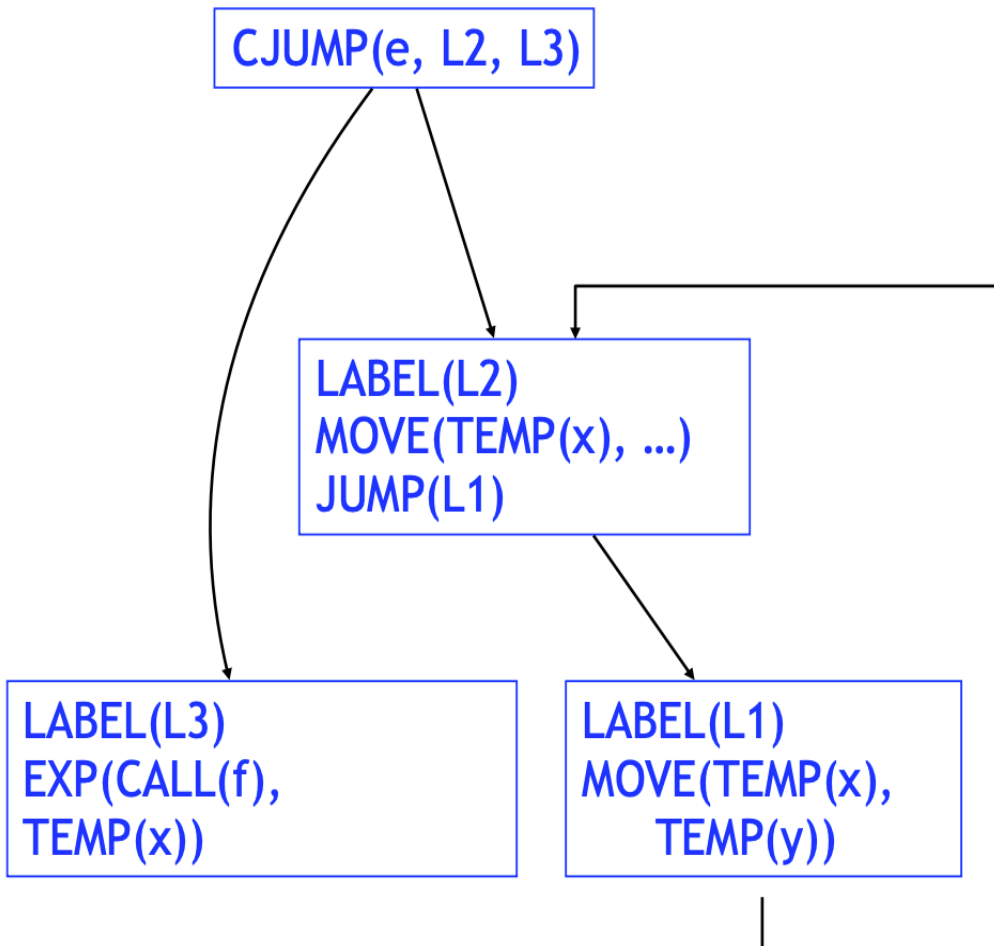
Key Concepts to Remember

Concept	Definition
Canonical tree	No SEQ/ESEQ; CALL only under EXP or MOVE(TEMP)
Commuting	s and e commute if executing s doesn't change e's value
Basic block	LABEL at start, JUMP/CJUMP at end, nothing else
Trace	A possible consecutive execution path through blocks
Trace covering	Partition of all blocks into traces
Fall-through	Next instruction executes without a branch



Thank you all for your attention

Example: Reordered and Optimized Code



CJUMP(e, L2, [L3])

JUMP(L3)

LABEL(L2)

MOVE(TEMP(x), TEMP(y) +
TEMP(z))

~~JUMP(L1)~~

LABEL(L1)

MOVE(TEMP(x), TEMP(y))

JUMP(L2)

LABEL(L3)

EXP(CALL(NAME(f)), TEMP(x))